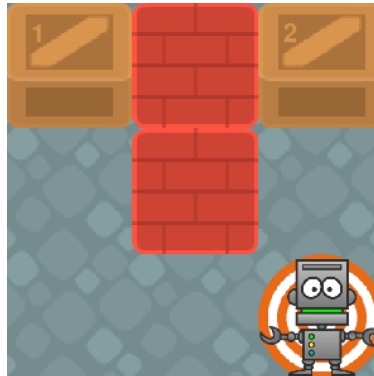


CS 7638: Artificial Intelligence for Robotics

Path Search: Smoothing Project - Summer 2026 - Deadline: Monday July 13th, Midnight AOE



Introduction

This PDF serves as a problem specification document, and thus, the information in this document is not referring to any details regarding any specific algorithm(s) to solve the problem. Its purpose is to define the problem along with the relevant details needed to solve it. It acts as a summary of the rules that are implemented in the testing suite code provided to you (which you can reference when you need further clarification). An interactive GUI mode (TEST_MODE) is also available for you to use (which you can use to validate your understanding of the specifications). Lastly, the example calculations and images in this PDF are also a good resource to use when looking for clarifications.

In part A of this project, you will implement a path search algorithm to guide a robot through a warehouse to pick up and drop off boxes to a designated drop zone area. In part B of this project, you will implement a path smoothing algorithm to allow a robot to find a more efficient route back to its charging station.

The template code provides 2 classes, one for each part of the project: `DeliveryPlanner_PartA` and `PathSmoothing_PartB`

- You may share code between part A and B
- Your submission will consist of a single file: `path_search.py`

Grading

- The weighting for each part is:
 - Part A = 50%
 - Part B = 50%
- Within each part there are 10 test cases, each test case is equally weighted.
- No assumptions should be made about any similarities between local and Gradescope test cases as they will be different. All test cases will adhere to the rules laid out in this PDF. Your solution does not need to handle test cases that fall outside the scope of these rules.

Part A (50%)

Your task is to pick up and deliver all boxes listed in the `todo` list. You will do this by providing a list of motions that the testing suite will execute in order to complete all the deliveries. Your algorithm should determine the optimal path to take while completing this task.

`DeliveryPlanner_PartA`'s constructor must take five arguments: `self`, `warehouse_viewer`, `dropzone_location`, `todo`, and `box_locations`. It must also have a method called `plan_delivery` that takes 2 arguments: `self` and `debug` (set to `False` by default).

`plan_delivery` will only be invoked once for each test case. The testing suite will not execute code while your function is creating a solution to deliver ALL boxes in the todo list. This means that your warehouse map will not automatically update during your function invocation, you will need to perform all updates to your warehouse map representation as needed for processing your solution.

Part A Input Specifications

`warehouse_viewer` will be a custom object used by the testing suite. For all intents and purposes, you can think of it as a list of `m` lists, each inner list containing `n` characters, corresponding to the layout of the warehouse. The warehouse is an `m x n` grid. `warehouse_viewer[i][j]` corresponds to the spot in the `i`th row and `j`th column of the warehouse, where the 0th row is the northern end of the warehouse and the 0th column is the western end [e.g., $(y,x) = (0,0)$ is in the upper left corner].

NOTE: In part A, your code will not know (nor should it depend on) the size of the warehouse (`n` and `m`). You are **only** allowed to use the `warehouse_viewer` object (`WarehouseAccess`) in the following 2 ways:

1. You may access a particular cell in the warehouse using `warehouse_viewer[i][j]`
2. You may overwrite the contents of a cell using `warehouse_viewer[i][j] = '<some symbol>'` (Note: This is only for your convenience if needed.)

If we find that your code is using any other methods/approaches to use or manipulate the warehouse object, **you will receive a 0 for part A.** (This may be done manually after the project is due.)

The goal in part A is not only to find an optimal solution, but to do so in an efficient manner. Efficiency here means that your algorithm should access (view) as few of the warehouse cells as possible. Accessing a particular cell more than once will NOT hurt your score as we will only tally the unique cells that your algorithm accesses for each test case. It is your responsibility to make sure you are not using any other way to glean information from the warehouse other than the 2 methods above. There are **some** precautions in place to help notify you when you are improperly using the warehouse object, however, these are **not** exhaustive. A few things to ask yourself below (If you answer yes to any of these, you are likely approaching the problem incorrectly and likely to receive a 0 for Part A.):

1. Are you attempting to determine the size of the warehouse?
2. Are you using the `len()` function on the warehouse or its rows/cols?
3. Are you attempting to access an entire row of the warehouse (rather than a particular cell)?
4. Are you accessing a private attribute (as indicated by a leading underscore) of the warehouse?
5. Are you iterating through the warehouse in any way?
6. Are you somehow gaining access to a warehouse cell's contents without it being counted towards the `viewed_cell_count`?
7. Are you attempting to copy the warehouse object/data in any way?
8. Are you printing the warehouse object?

NOTE: The box locations and the dropzone are counted as viewed cells because they are given to you as input.

The characters in each string will be one of the following:

`.` (period) = traversable space. The robot may enter from any adjacent space.

`#` (hash) = a wall/obstacle. The robot cannot enter (or exist at) this space. All warehouse maps in the test cases of Part A will be surrounded by walls.

`@` (dropzone) = the starting point for the robot and the space where all boxes must be delivered. The dropzone may be traversed like a `.` (period).

`[0-9a-zA-Z]` (any alphanumeric character) = a box. At most one of each alphanumeric character will be present in the warehouse (meaning there will be at most 62 boxes). A box on the ground may not be traversed, but if the robot is adjacent to the box, the robot can pick up the box. Once the box has been lifted, the space that the lifted box previously occupied now is traversable/functions as a `.` (period).

For example,

```
warehouse_viewer = ['#####',
                    '#1#2#',
                    '#.#.#',
                    '#..@#',
                    '#####']
```

is a 5x5 warehouse.

- The dropzone is at the warehouse cell in row 3, column 3.
- Box 1 is located in the warehouse cell in row 1, column 1.
- Box 2 is located in the warehouse cell in row 1, column 3.
- There are walls within the warehouse at cells (row 1, column 2) and (row 2, column 2) and around the warehouse.
- The remaining five warehouse cells (which includes the dropzone) are traversable spaces.

The argument `todo` is a list of alphanumeric characters giving the order in which the boxes must be delivered to the dropzone. For the above example, if `todo = ['1', '2']`, then the robot must first deliver box 1 to the dropzone and then deliver box 2 to the dropzone. No other boxes can be picked up until the robot successfully delivers the first box.

Part A Rules & Costs for Motions

- The robot may move in 8 directions (N, NE, E, SE, S, SW, W, NW)
- The robot may not move outside the warehouse. The warehouse does not “wrap” around (it is not cyclic).
- Two spaces are considered **adjacent** if they share an edge (orthogonally) or a corner (diagonally).
- The robot may pick up a box that is in any adjacent square.
- The robot may put a box down in an adjacent square, so long as the adjacent square is empty (`.` or `@`).
- The warehouse will be arranged so that it is always possible for the robot to move to the next box on the `todo` list without having to rearrange any other boxes. (Only necessary to drop boxes in the dropzone.)
- If a box is placed on the `@` space, it is considered delivered and is removed from the warehouse, thus the `@` space is still traversable after dropping a box on it.
- While holding a box, the robot may not pick up another box.
- There are 4 kinds of **motions** that the robot can take (below are the costs associated with each type):
 - `[cost]`: type
 - `[2]`: horizontal or vertical movement
 - `[3]`: diagonal movement
 - `[4]`: pick up box (regardless the direction)
 - `[2]`: put down box (regardless the direction)
- A robot attempting an illegal motion will incur both the cost for the requested illegal motion in addition to an illegal motion penalty of 100.
- A robot attempting an illegal motion will not actually move (remains in the original location).
- Illegal motions include:
 - attempting to move to a nonadjacent, nonexistent, or occupied space
 - attempting to pick up a nonadjacent or nonexistent box
 - attempting to pick up a box while already holding one (attempting to put down a box while not holding one)
 - attempting to put down a box on a nonadjacent, nonexistent, or occupied space. (This means the robot may not drop a box on the drop zone while the robot is occupying the drop zone.)

Part A Method Return Specifications

`plan_delivery` should return a list of moves that minimizes the total cost of completing the task. Each move should be a string formatted as follows:

- 'move {d}', where '{d}' is replaced by the direction the robot should move: "n", "ne", "e", "se", "s", "sw", "w", "nw"
- 'lift {x}', where '{x}' is replaced by the alphanumeric character of the box being picked up
- 'down {d}', where '{d}' is replaced by the direction the robot will put the box down

For example, using the values of `warehouse` and `todo` given previously (reproduced below):

```
warehouse_viewer = ['#####',
                    '#1#2#',
                    '#.#.#',
                    '#..@#',
                    '#####']

todo = ['1', '2']
```

`plan_delivery` might return the following:

```
['move w',
 'move nw',
 'lift 1',
 'move se',
 'down e',
 'move ne',
 'lift 2',
 'down s']
```

Part A Scoring

The testing suite will execute your plan and calculate the total cost: `student_cost`. The score for each test case will be calculated by: `benchmark_cost / student_cost`. The benchmark will be greater than or equal to the absolute minimum cost. You will receive a 0 in the following situations:

- your code views more warehouse cells than specified in the test case: `viewed_cell_count_threshold`
- your code takes longer than the prescribed time limit
- your method returns the wrong output format
- the boxes are not delivered in the correct order/not all boxes are delivered

Part B (50%)

Your task is to get your robot back to a charging station in the warehouse. You will do this by providing a list containing integer points corresponding to your smoothed path as well as a list of motions that the testing suite will execute in order to follow your smoothed path and efficiently arrive at the charging station. Your algorithm will have access to: (1) a non-smoothed path (“path”) starting at your robot location and ending at the charging station and (2) a list of current obstacles (“obs”). When performing path smoothing, the corresponding point (See “Corresponding Points” diagram p.9 for clarity.) on your smoothed path must stay within 3 units of the corresponding point on the original path to avoid permanent obstacles and safely navigate the warehouse. Additionally, in low power mode your robot cannot pick up boxes, so they become temporary obstacles (listed in “obs”) with which your robot must not collide. During low power mode, the robot is no longer running some stabilizing actuators, so in addition to not being able to pick up boxes, it can only safely make 45 degree turns without risking tipping over. Your algorithm should determine a smoothed path (judged against a benchmark path length), which the robot should take to reach its charging station safely (no risk of tipping over). The robot must occupy the charging station cell to begin charging.

`PathSmoothing_PartB`'s constructor must take five arguments: `self`, `warehouse`, `obs`, `path`, `robot_init`,

and `benchmark_path`. Where `warehouse` is the same as Part A, `obs` is a list of the temporary obstacles that must be avoided, `path` is a non-smoothed path that you must stay within 3 units of, `robot_init` is the initial starting point of the robot, and `benchmark_path` is the maximum length of an efficient smoothed path to receive full credit.

It must also have a method called `smooth_path` that takes 2 arguments: `self` and `debug` (set to `False` by default). Some starter debug code is included to print paths, compare your smoothed path length to the benchmark, and list moves to assist with your debugging. Feel free to modify this code in any way that helps with your debugging. Please make sure that any print statements are inside debug statements to avoid issues when submitting to Gradescope. `smooth_path` will only be invoked once for each test case. The testing suite will not execute code while your function is creating a smoothed path solution to travel to the charging station. This means that if you are using a warehouse map, it will not automatically update during your function invocation, so you will need to perform all updates to your warehouse map representation as necessary for processing your solution.

Part B Input Specifications

`_set_initial_state_from` will be a custom object that is available for Part B but is optional. This function is the same one used in Part A.

NOTE: In part B, there is no penalty for viewing all cells in your warehouse (unlike Part A). Depending on your solution, you may or may not need to use warehouse viewer (it is not required).

1. You may access a particular cell in the warehouse using `warehouse_viewer[i][j]` (corresponds to `i`th row and `j`th column)
2. You may overwrite the contents of a cell using `warehouse_viewer[i][j] = '<some symbol>'` (note that this is only for your convenience if needed)

The characters in each string will be one of the following:

`.` (period) = traversable space. The robot may enter from any adjacent space (orthogonal or diagonal).

`#` (hash) = an obstacle. The robot cannot enter (or exist at) this space. Unlike Part A, the warehouse maps in the test cases of Part B do not necessarily have exterior walls.

`@` (charging station) = the goal that the robot needs to reach to successfully recharge. The charging station may be traversed like a `.` (period).

Boxes only exist as obstacles in the “obs” list. There are no boxes that need to be picked up or delivered in Part B.

For example, in Test Case 1:

```
params = {'test_case': 1,
          'warehouse': ['....',
                        '....',
                        '....',
                        '@...',
                        '...#'],
          'obs': [[3, 3]],
          'path': [[0, 0], [1, 0], [2, 0], [2, 1]],
          'robot_init': [0, 0],
          'benchmark_path': 3}
```

is a 4x4 warehouse with $(y,x) = (0,0)$ at the top left corner of the map.

- The robot starts at $(0,0)$.
- The charging station (goal) is located at $(2,1)$.
- A single obstacle is located at $(3,3)$.
- The path that your robot needs to stay within 3 units for safety is 4 points: $[[0, 0], [1, 0], [2, 0], [2, 1]]$.
- There are no walls around the warehouse.

- The benchmark path length is 3 points, meaning that to get full credit for the test case your smoothed path must be 3 points or less.
- There are no boxes to deliver in Part B.

Part B Rules for Motions

Rules that are the same as Part A:

- The robot may move in 8 directions (N, NE, E, SE, S, SW, W, NW)
- The robot may not move outside the warehouse, which does not “wrap” around (it is not cyclic).
- Two spaces are considered **adjacent** if they share an edge (orthogonal) or a corner (diagonal).

Rules that are different from Part A:

- When your robot is charging, it occupies the charging station (@ space), thus the @ space is considered traversable (you will not be dropping a box on it for Part B).
- If the moves for your smoothed path would cause your robot to perform an illegal motion (colliding with an obstacle) or a motion that violates its constraints (no turns greater than 45 deg, no diverging more than 3 units from the given path) then your solution will be penalized. It is possible to receive partial credit.
- There are 2 kinds of **motions** that the robot can take (below are the costs associated with each type). You can use costs to assist your algorithm in obstacle avoidance and/or in logic pertaining to limiting turn radius to 45 deg or less. However, Part B solutions are not required to utilize these costs:
 - [cost]: type
 - [2]: horizontal or vertical movement
 - [3]: diagonal movement

Part B Method Return Specifications

def smooth_path should have 2 returns:

- **smooth_path** a list points that define your smoothed path [[y1, x1], [y2, x2], ... [yn, xn]]
- **moves** a list of moves that that your robot will need to make to follow your smoothed path.

Each move should be a string formatted as follows:

- 'move {d}', where '{d}' is replaced by the direction the robot should move: "n", "ne", "e", "se", "s", "sw", "w", "nw"
- Your robot just has to occupy the cell that the charging station is in. (It is an inductive charger.) There is no special move command for charging.

For example, for the values in Test Case 1, given previously (reproduced below):

```
params = {'test_case': 1,
          'warehouse': ['....',
                        '....',
                        '...@...',
                        '...#'],
          'obs': [[3, 3]],
          'path': [[0, 0], [1, 0], [2, 0], [2, 1]],
          'robot_init': [0, 0],
          'benchmark_path': 3}
```

smooth_path might return the following:

```
smooth_path = [[0, 0], [1, 0], [2, 1]]
moves = ['move s', 'move se']
```

Part B Scoring

There are 10 test cases, and each test case is equally weighted.

The testing suite will execute your plan and calculate the total score based on the following requirements (Meeting 2 of the requirements will give you 0.5 pt, while meeting all 3 will give you 1.0 pt):

- Each point on the smoothed path stays within 3 units of its corresponding point on the given safe path.
- Your robot's max turn angle is 45 deg or less.
- The length of your smoothed path is less than or equal to the `benchmark_path` length.

You will receive a 0 in the following situations:

- your moves fail to reach the charging station
- your moves cause a collision with an obstacle
- your `smooth_path` is equal to the input path
- your code takes longer than the prescribed time limit
- your method returns the wrong output format

Environment Test

Before changing `path_search.py`, test your environment using the following steps:

- 1) From the command line run: `python path_search.py`
 - A list of moves for Part A test case 1 will be printed
 - A `smooth_path` and list of moves will be printed for part B test case 1
- 2) From the command line run: `python testing_suite_PartA.py` [or B or full (A & B)]
 - A list of test cases and their score should show that test case 1 passed and the remaining failed.

Visualization Part A

There is an ASCII based visualization which will print the warehouse state and other important data to the console. This can be set by using the `VERBOSE_FLAG` in the testing suite files.

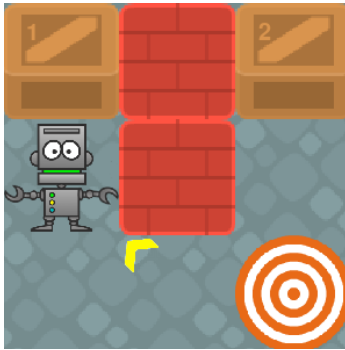
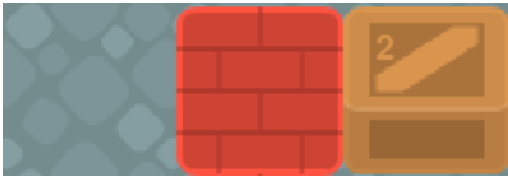
In addition, there is a GUI based visualization, set `VISUALIZE_FLAG=True`. You can change the GUI frame rate speed in the `visualizer.py` file. The 6 choices are `[1,2,3,4,5]` (slow to fast) and `[0]` which is MANUAL-PAUSE mode (this will not proceed to the next time step until you press the **space bar**). You can conveniently quit any test case by pressing the **Esc** (escape) key. An example demo video can be found at the following link: https://mediaspace.gatech.edu/media/t/1_onp8ge69

Part A's visualization will also indicate which cells your algorithm (did and did not) access during the search process. The cells with a dark overlay on them indicate cells that your algorithm did not access.

In `testing_suite_partA.py`, you can turn on `TEST_MODE` to control the robot with your keyboard (Note: NumLock may need to be turned off). This can be used to validate and build your understanding about the rules of the game. The controls are the following:

 q 7	 w 8	 e 9
 a 4	<not used> s 5	 d 6
 z 1	 x 2	 c 3

lift: shift + <key direction>
down: ctrl/% + <key direction>

Illegal Move Indication	Illegal “box” Lift Indication	Illegal Move Example
		
Successfully Moved Diagonally	Successfully Put Box Down	Successfully Lifted Box
		
Cells Accessed	Cells Not Accessed	
		

Visualization Part B

For Part B, the visualization is also turned on by setting VISUALIZE_FLAG=True.

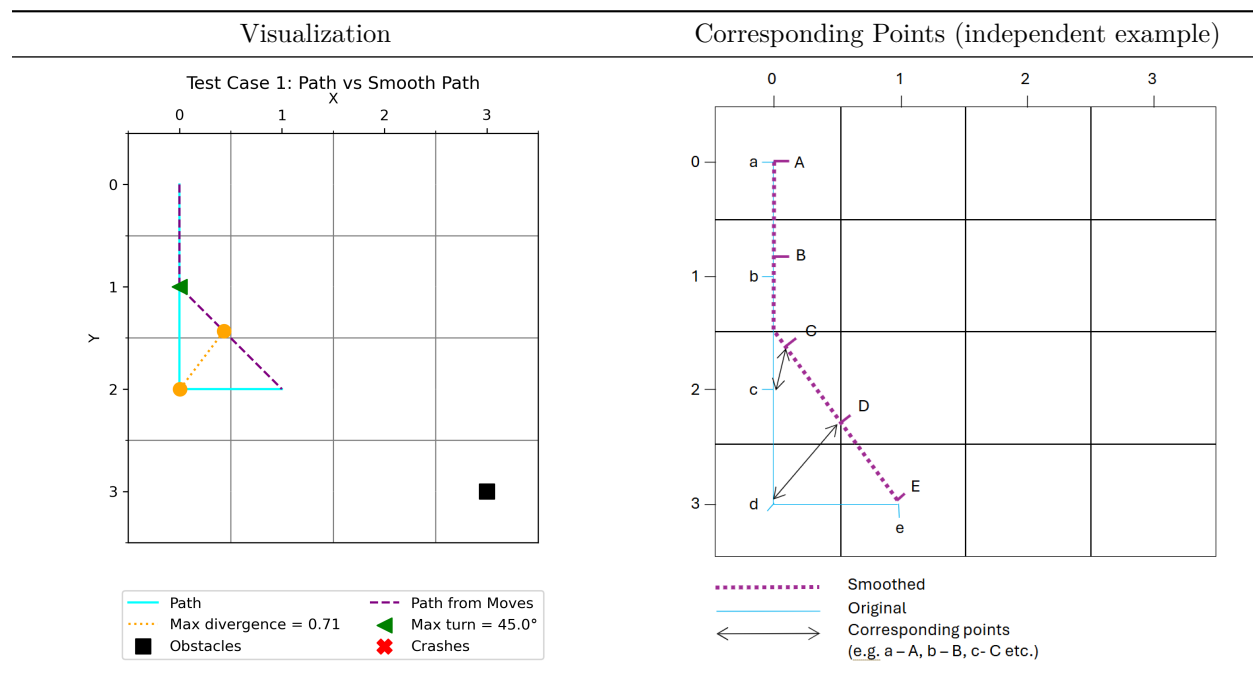
Turn on Visualizer

```

35  #####
36  # For visualization this flag can be set to True to display a GUI
37  # which could result in a timeout, but useful for debugging
38  # Note that enabling this will also enable DEBUGGING_SINGLE_PROCESS
39  #####
40  VISUALIZE_FLAG = True

```

The visualization for Part B will draw a plot of the warehouse with the original path, smooth_path, and obstacles as well as show grading parameters: max divergence between the 2 paths, max turn angle, and any collisions with obstacles.



The n th equidistant point on the original path is a corresponding point of the n th equidistant point on the smoothed path.

Development and Debugging

When developing and debugging here are some ideas that might prove helpful.

1. Test your algorithm using a single test case:

- You can run a single test case. For example, to run the first test case for Part A:

```
python testing_suite_partA.py PartATestCase.test_case_01
```

- Or you may comment out all but a single test case in the testing suite

2. If testing in a debugger, to allow breakpoints to work properly, there are some flags that can be set at the top of the testing_suite_part[A B].py files.

```

29  # For debugging this flag can be set to True to print state
30  # which could result in a timeout
31  # Note that enabling this will also enable DEBUGGING_SINGLE_PROCESS
32  VERBOSE_FLAG = True
33
34  # For visualization this flag can be set to True to display a GUI
35  # which could result in a timeout, but useful for debugging
36  # Note that enabling this will also enable DEBUGGING_SINGLE_PROCESS
37  VISUALIZE_FLAG = False
38
39  # For debugging set the time limit to a big number (like 600 or more)
40  # Note that enabling this will also enable DEBUGGING_SINGLE_PROCESS
41  TIME_LIMIT = 600 # 5 seconds
42
43  # If your debugger does not handle multiprocess debugging very easily
44  # then when debugging set the following flag true.
45  DEBUGGING_SINGLE_PROCESS = True

```

- Set the TIME_OUT to a very large value (like 600 seconds)
- Set DEBUGGING_SINGLE_PROCESS = True (this disables multiprocessing, which messes up most debuggers)
- Set VERBOSE_FLAG = True
 - Provides a simple console based visualization
 - Provides line numbers for any syntax errors that occur
 - If exceptions are raised, provides detailed stack trace
 - For Part B, will print paths and values of calculations used in grading